Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 1
з дисципліни: «Теорія алгоритмів»
Тема: «Квадратичні алгоритми сортування»

Варіант № 22

Виконав:
Студент групи АІ-254
Церковний О. Г.
Перевірили:
Смик С. Ю.
Арсірій О.О.

Одеса 2026

Мета роботи: Метою виконання лабораторної роботи є набуття практичних навичок із проектування, реалізації, тестування та аналізу алгоритмів сортування вибором та вставкою. За рахунок виконання трасування та тестування студент має практично довести квадратичну часову складність та стійкість вивчених алгоритмів, а також порівняти ці алгоритми за кількістю базових операцій.

Завдання:
1. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм сортування вибором.
2. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм сортування вставками
3. Порівняти обчислювальну складність алгоритмів сортування вибором та вставками за базовими операціями.
Варіант 22. Послідовність: 19, 75, 43, 31, 5, 66, 62, 34, 76

1. Сортування вибором

Лістинг 1.1 – Псевдокод алгоритму сортування вибором.

```
Алгоритм Selection_sort (A [0..n - 1])
// Сортування масиву методом вибору
// Вхідні дані: Масив A [0.. n - 1] елементів, що впорядковуються
// Вихідні дані: Масив A [0.. n - 1], відсортований у неспадному порядку
for i← 0 to n-2 do // зовнішній цикл
{
min ← i
for j ← i+1 to n -1 do // внутрішній цикл
if A[j] < A[min]
min← j
B ← A[i] // Обмін A[i] та A [min]
A[i]← A[min]
A[min]← B
}
```

Лістинг 1.2 – Python-код алгоритму сортування вибором.

```
def selection_sort(arr):
    n = len(arr)
    comparisons = 0
    assignments = 0

    # Зовнішній цикл
    # від 0 до n-2 (n-1 проходів)
    for i in range(n - 1):

        # Припускаємо, що поточний елемент є мінімальним
        min_index = i
        assignments += 1

        # Внутрішній цикл шукає найменший елемент в решті списку
        for j in range(i + 1, n):
            comparisons += 1

            if arr[j] < arr[min_index]:
                min_index = j
```

```
                assignments += 1 # Присвоєння змінній min_index

        # Обмін елементів, якщо знайдено новий мінімальний
        comparisons += 1 # Операція порівняння

        if min_index != i:
        # Обмін елементів
            arr[i], arr[min_index] = arr[min_index], arr[i]
            assignments += 3

    return arr, comparisons, assignments


# Приклад використання
my_list = [19, 75, 43, 31, 5, 66, 62, 34, 76]

sorted_list, comps, assigs = selection_sort(my_list.copy())
   # Використання .copy(), щоб не змінювати оригінал
print("Оригінальний список:", my_list)
print("Відсортований список:", sorted_list)
print(f"Кількість порівнянь: {comps}")
print(f"Кількість присвоєнь: {assigs}")
```

Таблиця 1.1 Результати чисельного моделювання та трасування алгоритму сортування вибором.

Чисельне моделювання псевдокоду Лістинг 1.1	Трасування Python-коду Лістинг 1.2
Вихідні дані A=[19, 75, 43, 31, 5, 66, 62, 34, 76] n=9 (for i) 1. i=0; min=0; j=0+1=1; A[1]<A[0] (75<19) — ні 1. j=2; A[2]<A[0] (43<19) — ні 2. j=3; A[3]<A[0] (31<19) — ні 3. j=4; A[4]<A[0] (5<19) — min=4 4. j=5; A[5]<A[4] (66<5) — ні 5. j=6; A[6]<A[4] (62<5) — ні 6. j=7; A[7]<A[4] (34<5) — ні 7. j=8; A[8]<A[4] (76<5) — ні (end for j) b=A[0] A[0]=A[4] A[4]=b A[0]=5 A[4]=19 A=[5, 75, 43, 31, 19, 66, 62, 34, 76] 2. i=1; min=1; j=1+1=2; A[2]<A[1] (43<75) — min=2 1. j=3; A[3]<A[2] (31<43) — min=3 2. j=4; A[4]<A[3] (19<31) — min=4 3. j=5; A[5]<A[4] (66<19) — ні 4. j=6; A[6]<A[4] (62<19) — ні 5. j=7; A[7]<A[4] (34<19) — ні 6. j=8; A[8]<A[4] (76<19) — ні	Початковий масив: [19, 75, 43, 31, 5, 66, 62, 34, 76] ---------- Ітерація 0: Поточний елемент (для обміну): arr[0] = 19 Шукаємо мінімальний елемент у частині: [19, 75, 43, 31, 5, 66, 62, 34, 76] Знайдено мінімальний елемент: arr[4] = 5 Обмін arr[0] (19) і arr[4] (5) Масив після ітерації 0: [5, 75, 43, 31, 19, 66, 62, 34, 76] ---------- Ітерація 1: Поточний елемент (для обміну): arr[1] = 75 Шукаємо мінімальний елемент у частині: [75, 43, 31, 19, 66, 62, 34, 76] Знайдено мінімальний елемент: arr[4] = 19 Обмін arr[1] (75) і arr[4] (19)

(end for j) b=A[1] A[1]=A[4] A[4]=b A[1]=19 A[4]=75 A=[5, 19, 43, 31, 75, 66, 62, 34, 76] 3. i=2; min=2; j=2+1=3; A[3]<A[2] (31<43) — min=3 1. j=4; A[4]<A[3] (75<31) — ні 2. j=5; A[5]<A[3] (66<31) — ні 3. j=6; A[6]<A[3] (62<31) — ні 4. j=7; A[7]<A[3] (34<31) — ні 5. j=8; A[8]<A[3] (76<31) — ні (end for j) b=A[2] A[2]=A[3] A[3]=b A[2]=31 A[3]=43 A=[5, 19, 31, 43, 75, 66, 62, 34, 76] 4. i=3; min=3; j=3+1=4; A[4]<A[3] (75<43) — ні 1. j=5; A[5]<A[3] (66<43) — ні 2. j=6; A[6]<A[3] (62<43) — ні 3. j=7; A[7]<A[3] (34<43) — min=7 4. j=8; A[8]<A[7] (76<34) — ні (end for j) b=A[3] A[3]=A[7] A[7]=b A[3]=34 A[7]=43 A=[5, 19, 31, 34, 75, 66, 62, 43, 76] 5. i=4; min=4; j=4+1=5; A[5]<A[4] (66<75) — min=5 1. j=6; A[6]<A[5] (62<66) — min=6 2. j=7; A[7]<A[6] (43<62) — min=7 3. j=8; A[8]<A[7] (76<43) — ні (end for j) b=A[4] A[4]=A[7] A[7]=b A[4]=43 A[7]=75 A=[5, 19, 31, 34, 43, 66, 62, 75, 76] 6. i=5; min=5; j=5+1=6; A[6]<A[5] (62<66) — min=6 1. j=7; A[7]<A[6] (75<62) — ні 2. j=8; A[8]<A[6] (76<62) — ні (end for j) b=A[5] A[5]=A[6] A[6]=b A[5]=62 A[6]=66 A=[5, 19, 31, 34, 43, 62, 66, 75, 76] 7. i=6; min=6; j=6+1=7; A[7]<A[6] (75<66) — ні 1. j=8; A[8]<A[6] (76<66) — ні	Масив після ітерації 1: [5, 19, 43, 31, 75, 66, 62, 34, 76] ---------- Ітерація 2: Поточний елемент (для обміну): arr[2] = 43 Шукаємо мінімальний елемент у частині: [43, 31, 75, 66, 62, 34, 76] Знайдено мінімальний елемент: arr[3] = 31 Обмін arr[2] (43) і arr[3] (31) Масив після ітерації 2: [5, 19, 31, 43, 75, 66, 62, 34, 76] ---------- Ітерація 3: Поточний елемент (для обміну): arr[3] = 43 Шукаємо мінімальний елемент у частині: [43, 75, 66, 62, 34, 76] Знайдено мінімальний елемент: arr[7] = 34 Обмін arr[3] (43) і arr[7] (34) Масив після ітерації 3: [5, 19, 31, 34, 75, 66, 62, 43, 76] ---------- Ітерація 4: Поточний елемент (для обміну): arr[4] = 75 Шукаємо мінімальний елемент у частині: [75, 66, 62, 43, 76] Знайдено мінімальний елемент: arr[7] = 43 Обмін arr[4] (75) і arr[7] (43) Масив після ітерації 4: [5, 19, 31, 34, 43, 66, 62, 75, 76] ---------- Ітерація 5: Поточний елемент (для обміну): arr[5] = 66 Шукаємо мінімальний елемент у частині: [66, 62, 75, 76]

(end for j) b=A[6] A[6]=A[6] A[6]=b A[6]=66 A[6]=66 A=[5, 19, 31, 34, 43, 62, 66, 75, 76] 8. i=7; min=7; j=7+1=8; A[8]<A[7] (76<75) — ні (end for j) b=A[7] A[7]=A[7] A[7]=b A[7]=75 A[7]=75 A=[5, 19, 31, 34, 43, 62, 66, 75, 76] (end for i)	Знайдено мінімальний елемент: arr[6] = 62 Обмін arr[5] (66) і arr[6] (62) Масив після ітерації 5: [5, 19, 31, 34, 43, 62, 66, 75, 76] ---------- Ітерація 6: Поточний елемент (для обміну): arr[6] = 66 Шукаємо мінімальний елемент у частині: [66, 75, 76] Знайдено мінімальний елемент: arr[6] = 66 Обмін не потрібен Масив після ітерації 6: [5, 19, 31, 34, 43, 62, 66, 75, 76] ---------- Ітерація 7: Поточний елемент (для обміну): arr[7] = 75 Шукаємо мінімальний елемент у частині: [75, 76] Знайдено мінімальний елемент: arr[7] = 75 Обмін не потрібен Масив після ітерації 7: [5, 19, 31, 34, 43, 62, 66, 75, 76] ---------- Сортування завершено. Фінальний відсортований масив: [5, 19, 31, 34, 43, 62, 66, 75, 76] Загальна кількість порівнянь: 44 Загальна кількість присвоєнь: 36

Результати виконання Python-коду показано далі:

- Оригінальний список: [19, 75, 43, 31, 5, 66, 62, 34, 76]
- Відсортований список: [5, 19, 31, 34, 43, 62, 66, 75, 76]
- Кількість порівнянь: 44
- Кількість присвоєнь: 36

Кінцевий результат
Початковий масив:

[19, 75, 43, 31, 5, 66, 62, 34, 76]
Відсортований масив:
[5, 19, 31, 34, 43, 62, 66, 75, 76]

Сумарна кількість базових операцій:
44+36=80

2. Сортування вставками

Лістинг 1.3 – Псевдокод алгоритму сортування вставками.

```
Insertion_Sort

for j = 2 to A.length do
    key = A[j]
    i = j - 1
    while (i > 0 and A[i] > key) do
        A[i + 1] = A[i]
        i = i - 1
    end while
    A[i + 1] = key
end
```

Лістинг 1.4 – Python-код алгоритму сортування вставками.

```
def insertion_sort(arr):
    n = len(arr)
    comparisons = 0
    assignments = 0

    for i in range(1, n):
        key = arr[i]
        assignments += 1

        j = i - 1
        assignments += 1

        while j >= 0 and arr[j] > key:
            comparisons += 1

            arr[j + 1] = arr[j]
            assignments += 1

            j -= 1
            assignments += 1

        if j >= 0:
            comparisons += 1

        arr[j + 1] = key
        assignments += 1

    return arr, comparisons, assignments


my_list = [19, 75, 43, 31, 5, 66, 62, 34, 76]

sorted_list, comps, assigs = insertion_sort(my_list.copy())

print("Оригінальний список:", my_list)
```

```
print("Відсортований список:", sorted_list)
print(f"Кількість порівнянь: {comps}")
print(f"Кількість присвоєнь: {assigs}")
```

Таблиця 1.2 Результати чисельного моделювання та трасування алгоритму сортування вставками.

Чисельне моделювання псевдокоду Лістинг 1.3	Трасування Python-коду Лістинг 1.4

Вихідні дані A=[19, 75, 43, 31, 5, 66, 62, 34, 76] n=9;
A.length=9 (for j)
1. j=2; key=A[2]=75
i=2-1=1
i>0 (True); A[1]>key (19>75 - False);
A[2]=key=75
A=[19, 75, 43, 31, 5, 66, 62, 34, 76]
 	
2. j=3; key=A[3]=43; i=3-1=2; i>0 (True); A[2]>key (75>43 — True) →
A[3]=A[2]=75;
A=[19, 75, 75, 31, 5, 66, 62, 34, 76]
i=2-1=1;
i>0 (True);
A[1]>key (19>43 — False);
A[2]=key=43
A=[19, 43, 75, 31, 5, 66, 62, 34, 76]
 	
3. j=4; key=A[4]=31; i=4-1=3
i>0 (True)
A[3]>key (75>31 — True) →
A[4]=A[3]=75
A=[19, 43, 75, 75, 5, 66, 62, 34, 76]
i=3-1=2;
i>0 (True)
A[2]>key (43>31 — True) →
A[3]=A[2]=43
A=[19, 43, 43, 75, 5, 66, 62, 34, 76]
i=2-1=1; i>0 (True)
A[1]>key (19>31 — False);
A[2]=key=31
A=[19, 31, 43, 75, 5, 66, 62, 34, 76]
 	
4. j=5; key=A[5]=5
i=5-1=4
	Початковий масив: [19, 75, 43, 31, 5, 66, 62, 34, 76]
	---------- 	
	Ітерація 1:
	Елемент для вставки (key): 75
	Відсортована частина: [19]
	Порівняння: 19 > 75. False. Цикл завершено.
	Вставка 75 на позицію 1.
	Масив після ітерації 1:
	[19, 75, 43, 31, 5, 66, 62, 34, 76]
	---------- 	
	Ітерація 2:
	Елемент для вставки (key): 43
	Відсортована частина: [19, 75]
	Порівняння: 75 > 43. True. Зсуваємо 75 вправо.
	Порівняння: 19 > 43. False. Цикл завершено.
	Вставка 43 на позицію 1.
	Масив після ітерації 2:
	[19, 43, 75, 31, 5, 66, 62, 34, 76]
	---------- 	
	Ітерація 3: Елемент для вставки (key): 31
	Відсортована частина: [19, 43, 75]
	Порівняння: 75 > 31. True. Зсуваємо 75 вправо.
	Порівняння: 43 > 31. True. Зсуваємо 43 вправо
	Порівняння: 19 > 31. False. Цикл завершено.
	Вставка 31 на позицію 1.
	Масив після ітерації 3:
	[19, 31, 43, 75, 5, 66, 62, 34, 76]
	---------- 	

i>0 (True); A[4]>key (75>5 — True) → A[5]=A[4]=75 A=[19, 31, 43, 75, 75, 66, 62, 34, 76]; i=4-1=3; i>0 (True) A[3]>key (43>5 — True) → A[4]=A[3]=43 A=[19, 31, 43, 43, 75, 66, 62, 34, 76] i=3-1=2 i>0 (True) A[2]>key (31>5 — True) → A[3]=A[2]=31 A=[19, 31, 31, 43, 75, 66, 62, 34, 76]; i=2-1=1; i>0 (True) A[1]>key (19>5 — True) → A[2]=A[1]=19 A=[19, 19, 31, 43, 75, 66, 62, 34, 76] i=1-1=0; i>0 (False) A[1]=key=5 A=[5, 19, 31, 43, 75, 66, 62, 34, 76] 5. j=6 key=A[6]=66; i=6-1=5 i>0 (True) A[5]>key (75>66 — True) → A[6]=A[5]=75 A=[5, 19, 31, 43, 75, 75, 62, 34, 76] i=5-1=4; i>0 (True) A[4]>key (43>66 — False) A[5]=key=66 A=[5, 19, 31, 43, 66, 75, 62, 34, 76] 6. j=7; key=A[7]=62; i=7-1=6; i>0 (True); A[6]>key (75>62 — True) → A[7]=A[6]=75; A=[5, 19, 31, 43, 66, 75, 75, 34, 76]; i=6-1=5; i>0 (True); A[5]>key (66>62 — True) → A[6]=A[5]=66; A=[5, 19, 31, 43, 66, 66, 75, 34, 76]; i=5-1=4; i>0 (True); A[4]>key (43>62 — False); A[5]=key=62; A=[5, 19, 31, 43, 62, 66, 75, 34, 76] 7. j=8; key=A[8]=34; i=8-1=7; i>0 (True); A[7]>key (75>34 — True) → A[8]=A[7]=75	Ітерація 4: Елемент для вставки (key): 5; Відсортована частина: [19, 31, 43, 75]; Порівняння: 75 > 5. True. Зсуваємо 75 вправо. Порівняння: 43 > 5. True. Зсуваємо 43 вправо. Порівняння: 31 > 5. True. Зсуваємо 31 вправо. Порівняння: 19 > 5. True. Зсуваємо 19 вправо. Досягнуто початку масиву. Цикл завершено Вставка 5 на позицію 0. Масив після ітерації 4: [5, 19, 31, 43, 75, 66, 62, 34, 76] ---------- Ітерація 5: Елемент для вставки (key): 66 Відсортована частина: [5, 19, 31, 43, 75] Порівняння: 75 > 66. True. Зсуваємо 75 вправо. Порівняння: 43 > 66. False. Цикл завершено. Вставка 66 на позицію 4. Масив після ітерації 5: [5, 19, 31, 43, 66, 75, 62, 34, 76] ---------- Ітерація 6: Елемент для вставки (key): 62 Відсортована частина: [5, 19, 31, 43, 66, 75] Порівняння: 75 > 62. True. Зсуваємо 75 вправо. Порівняння: 66 > 62. True. Зсуваємо 66 вправо. Порівняння: 43 > 62. False. Цикл завершено. Вставка 62 на позицію 4; Масив після ітерації 6: [5, 19, 31, 43, 62, 66, 75, 34, 76] ---------- Ітерація 7: Елемент для вставки (key): 34

A=[5, 19, 31, 43, 62, 66, 75, 75, 76];
i=7-1=6; i>0 (True);
A[6]>key (66>34 — True) →
A[7]=A[6]=66;
A=[5, 19, 31, 43, 62, 66, 66, 75, 76];
i=6-1=5;
i>0 (True);
A[5]>key (62>34 — True) →
A[6]=A[5]=62;
A=[5, 19, 31, 43, 62, 62, 66, 75, 76];
i=5-1=4;
i>0 (True);
A[4]>key (43>34 — True) →
A[5]=A[4]=43;
A=[5, 19, 31, 43, 43, 62, 66, 75, 76]; i=4-1=3; i>0 (True);
A[3]>key (31>34 — False);
A[4]=key=34; A=[5, 19, 31, 34, 43, 62, 66, 75, 76]
 	
8. j=9; key=A[9]=76;
i=9-1=8;
i>0 (True);
A[8]>key (75>76 — False);
A[9]=key=76;
A=[5, 19, 31, 34, 43, 62, 66, 75, 76]
(End for j)
	Відсортована частина: [5, 19, 31, 43, 62, 66, 75]
	Порівняння: 75 > 34. True. Зсуваємо 75 вправо.
	Порівняння: 66 > 34. True. Зсуваємо 66 вправо.
	Порівняння: 62 > 34. True. Зсуваємо 62 вправо.
	Порівняння: 43 > 34. True. Зсуваємо 43 вправо;
	Порівняння: 31 > 34. False. Цикл завершено.
	Вставка 34 на позицію 3.
	Масив після ітерації 7:
	[5, 19, 31, 34, 43, 62, 66, 75, 76]
	---------- 	
	Ітерація 8: Елемент для вставки (key): 76.
	Відсортована частина: [5, 19, 31, 34, 43, 62, 66, 75];
	Порівняння: 75 > 76. False. Цикл завершено.
	Вставка 76 на позицію 8; Масив після ітерації 8:
	[5, 19, 31, 34, 43, 62, 66, 75, 76]
	---------- 	
	Сортування завершено.
	Фінальний відсортований масив:
	[5, 19, 31, 34, 43, 62, 66, 75, 76]
	Загальна кількість порівнянь: 21
	Загальна кількість присвоєнь: 52

3. Аналіз обчислювальної складності алгоритмів сортування вибором та вставками

Таблиця 1.3. Порівняння обчислювальної складності алгоритмів сортування вибором та вставками

Назва алгоритму	Теоретичні відомості: порівняння	Теоретичні відомості: присвоювання	Результати експерименту: порівняння	Результати експерименту: присвоювання
Selection Sort	$O(n^2)$	$O(n)$	44	36
Insertion Sort	$O(n)$ (Best); $O(n^2)$ (Worst, Average)	$O(n)$ (Best); $O(n^2)$ (Worst, Average)	21	52

Висновок: У ході лабораторної роботи було реалізовано та досліджено алгоритми сортування вибором та вставками для заданої послідовності варіанта 22:
[19, 75, 43, 31, 5, 66, 62, 34, 76].
У результаті роботи обидва алгоритми сформували однакову відсортовану послідовність:
[5, 19, 31, 34, 43, 62, 66, 75, 76].
Для сортування вибором було виконано 44 порівняння та 36 присвоювань, а для сортування вставками - 21 порівняння та 52 присвоювання. Загальна кількість базових операцій становила відповідно 80 та 73.
Отримані результати підтверджують залежність кількості базових операцій від структури вхідної послідовності. Для n=9 (n^2=81) сумарна кількість операцій обох алгоритмів (80 та 73) відповідає квадратичній складності O(n^2). Сортування вибором завжди виконує однакову кількість порівнянь незалежно від вхідних даних: $n(n-1)/2 = 36$ порівнянь елементів A[j] < A[min] плюс $n-1 = 8$ службових перевірок min_index != i (їх враховує методичний приклад), разом 44 і мінімізує кількість присвоювань (не більше одного обміну на прохід), тоді як сортування вставками виконує менше порівнянь (21), але більше присвоювань (52) через зсуви елементів; на майже відсортованих даних воно наближається до O(n). Сортування вставками в наведеній реалізації є стійким (умова arr[j] > key строга, рівні елементи не змінюють взаємного порядку), а сортування вибором у такій реалізації є нестійким, оскільки обмін віддалених елементів може змінити порядок рівних ключів.

Посилання на GitHub: https://github.com/TserkovniyAlex/ta-lab1-22.git